Master's Thesis Computing Science

Radboud University Nijmegen

Evaluating Data Stream Write Performance in Lakehouse Systems

Author:
Vinícius Ribeiro Machado Schmidt
s1123702

University supervisor:
prof. dr. ir. Djoerd Hiemstra
hiemstra@cs.ru.nl

Company supervisor:
Chrystel Philipsen
chrystel.philipsen@sogeti.com
Marcel de Wit
marcel.de.wit@sogeti.com

Second assessor:
dr. Johannes Textor
johannes.textor@ru.nl

February 10, 2026

Abstract

Contents

Chapter 1

Introduction

1.1 Motivation and research questions

Many modern data storage architectures rely on so-called *data lakes*, which consist of (cloud) object storage, e.g. Amazon S3[1], Azure Blob Storage[2], for storing large amounts of (un)structured and semi-structured data in a centralized place [41]. Part of the data can then be copied into data warehouses through a process called "extract, transform, load" (ETL), so that it can be used for further downstream analytics tasks [27]. Although this two-tier architecture has become widespread among many enterprises, it has a few drawbacks. ETL pipelines used to load data into warehouses can be time-consuming and error-prone, causing the data in the warehouse to be outdated and sometimes inconsistent [32]. Additionally, maintaining the two systems is expensive and redundant, since part of the data is stored in both the data lake and the warehouse [27]. Finally, the flexibility of storing various file formats in the same place comes with the cost of managing different file versions and metadata, which with time can transform the data lake into a "data swamp" [8].

Following the limitations of data lakes and the two-tier architecture, Michael Armbrust et al. [27] proposed a new data storage architecture called *data lakehouse*. A lakehouse, also referred to as an open table format, builds on top of data lakes by structuring data into tables that are stored in immutable columnar formats, e.g. Apache Parquet[3]. These tables are managed with centralized metadata files that keep track of the current schema and the different versions of the table, all while ensuring that updates to tables occur in ACID [18] transactions [27]. Furthermore, the use of open formats for storage gives users control over their data, in contrast to proprietary formats of data warehouses [8], [27], [32]. These characteristics

[1] `https://aws.amazon.com/s3/`
[2] `https://azure.microsoft.com/en-us/products/storage/blobs/`
[3] `https://parquet.apache.org/`

allow analytical tasks to be performed directly on the lakehouse tables, thus eliminating the need of having a separate data warehouse, thus eliminating the need for a two-tier architecture.

Although open table formats have gained popularity since their introduction, some aspects of lakehouses have led to important questions in recent works. For instance, Paras Jain et al. [29] suggest that performing high frequent insertions into lakehouse tables can be challenging, because every insertion creates a new data file and requires updating the metadata files as well. Further, inserts in Delta Lake, a popular lakehouse format, can have up to hundreds of milliseconds of latency due to its reliance on object stores, which limits the throughput of streaming workloads, as mentioned in [7]. By contrast, the recent introduction of DuckLake [26] brought significant changes on how to handle write transactions to tables, while still ensuring ACID transactions. This is all thanks to the use of a traditional database management system (DBMS), e.g. PostgreSQL[4], for handling table metadata. Interestingly, no work has been done yet to evaluate the throughput of open table formats when performing data stream insertions. Therefore, this thesis will fill that gap, by focusing on the following research question:

"*How can we evaluate the throughput of data streams in lakehouse systems?*"

In addition, the following sub-questions will be answered

1. What are the current state-of-the-art benchmarks for lakehouses?
2. What are common patterns used in benchmarks for stream processing systems?
3. How can we ensure our benchmark is actually fair?
4. What is the difference in performance of current lakehouse systems when evaluated with our benchmark?
   (a) What is the performance gain of the "data inlining" feature of DuckLake in comparison to other lakehouse formats?

1.2 Contributions

1.3 Thesis structure

The rest of this thesis is structured as follows: Chapter 2 goes over the relevant background for this thesis, such as the different lakehouse technologies, how they are implemented and relevant features for this research.

[4] `https://www.postgresql.org/`

Chapter 3 talks about the existing work on benchmarking lakehouses and the different metrics employed by these benchmarks and the systems that they evaluate. Chapter 4 describes the methodology employed to develop our own benchmark and the reasoning behind those decisions. We also describe our experimental setup here. Chapter 5 present the results of our experiments, which will be discussed in Chapter 6. Finally, Chapter 7 will conclude this thesis by looking back at the proposed research questions, while proposing possible directions for future work.

Chapter 2

Background

Before we can develop our benchmark, we need to first understand how lakehouses work and which characteristics are relevant to the goal of this thesis. We will look at Iceberg and Delta Lake (Sections 2.1 and 2.2), which are the two most popular and more mature lakehouse formats based on GitHub stars count and initial release date respectively [22], [37]. Moreover, we will look at DuckLake, which brings innovative and relevant features for our research (discussed in Section 2.3), despite it still being in version 0.3.

This chapter will also go over the definitions of data stream and throughput, as well as guidelines on performing fair benchmarking.

2.1 Iceberg

Iceberg was developed in 2017 at Netflix and in 2018 it was released as an open-source project within the Apache Software Foundation[1].

2.1.1 Architectural design

Data layer

At the base of Iceberg's architecture, there is a data layer, which consists of data files that constitute the Iceberg table. They can be stored in one of three formats: Parquet, ORC[2] or Avro[3], with Parquet being the most common due to its widespread adoption and performance gains [24].

Metadata layer

Besides the data layer, there is the metadata layer, which keeps track of the data files locations, the version history of the table, as well as its current

[1] `https://www.apache.org/`
[2] `https://orc.apache.org/`
[3] `https://avro.apache.org/`

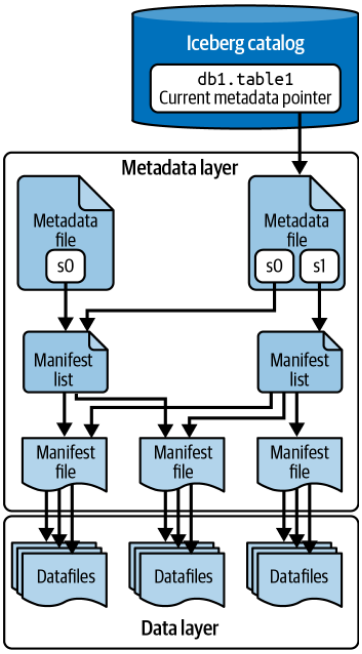

Figure 2.1: Architecture of an Iceberg table, as presented in [24].

schema. The files in the metadata layer are:

- **Manifest files**: Stored in Avro format, these files store pointers to multiple data files, as well as metadata about them, such as the minimum and maximum values of a file's columns and the number of records [24]. The metadata in manifest files is used for filtering data files during read operations, which helps improve read performance [24].
- **Manifest lists**: Manifest lists are another set of Avro files that individually point to a collection of manifest files. Together, the manifest files referenced by a manifest list constitute a specific snapshot, i.e. version, of an Iceberg table [24].
- **Metadata files**: These are JSON files that contain schema information about the table, as well as the snapshot history and a pointer to the current snapshot of that table [24]. A new metadata file is created after every update to the table, marking the creation of a new snapshot. Figure 2.1 depicts the architecture of an Iceberg table, where the newest metadata file has a pointer to both snapshots "s0" and "s1".

Catalog

In order to manage Iceberg tables, a catalog server is used. The goal of the catalog is to point to the latest metadata file of a table, as well as ensure that

this pointer is updated atomically [23], [24]. Consequently, all interactions with an Iceberg table must go through the catalog server. This avoids inconsistencies when clients are concurrently writing to the same table.

Although many technologies can be used as catalog, e.g. AWS Glue[4] and Hive Metastore[5], the developers of Iceberg provide a REST API spec [4] as an attempt to unify catalog access across different engines and languages. One prominent implementation of the Iceberg REST catalog is Apache Polaris[6], which uses a PostgreSQL database to store the latest metadata pointer.

2.1.2 Writing to Iceberg tables

Writing data to an Iceberg table, e.g. via `INSERT`, involves the following steps [24]:

1. First, the engine checks the catalog to get a pointer to the current metadata file, so that it can read the current table schema.

2. New data file(s) are written using the chosen storage format for the table.

3. The engine will then write the manifest file(s) that will contain metadata about the new data file(s), as well as a pointer to them.

4. Next, a new manifest list file is created, which will point to the new manifest file(s). In addition, existing manifest files are added to this new list, which together will correspond to the new snapshot of the table.

5. The engine writes a new metadata file which contains a pointer to the new snapshot (manifest list), as well as a history of all previous snapshots.

6. Finally, following the principle of optimistic concurrency [23], the engine will assume the table has not changed since the start of the operation and will try to atomically update the catalog to point to the new metadata file. However, if a concurrent write has created a new snapshot in the meantime, changing the metadata pointer fails and the engine has to retry. Iceberg tries to structure changes in such a way to minimize the cost of retries [3].

[4] `https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html`
[5] `https://hive.apache.org/`
[6] `https://polaris.apache.org/`

2.2 Delta Lake

Delta Lake was introduced in 2020 by Armbrust et al. [7]. It was originally developed at Databricks[7] and it was open-sourced in 2022 [35].

2.2.1 Architectural design

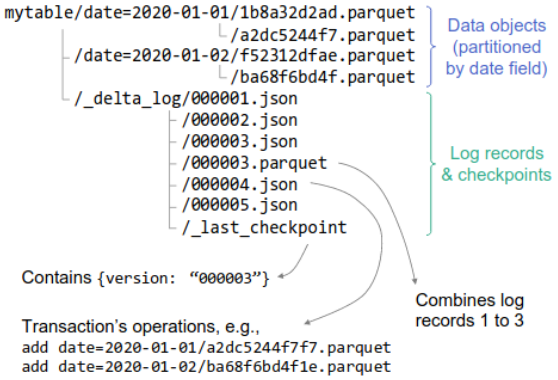

Figure 2.2: The architecture of a Delta Lake table, as presented in [7].

Data files

At its core, a Delta table consists of a directory containing data files, which are stored in the Parquet format. As shown if Figure 2.2, this data can be partitioned to improve efficiency of read queries, but this is beyond the scope of this thesis.

Delta log

The metadata of a Delta table is stored in a subdirectory, called "delta log". This directory contains JSON files, each named in increasing order according to the table version they correspond to, e.g. `00001.json`, `00002.json`, `00003.json`, etc. The log files themselves consist of actions, e.g. `add` and `remove`, that were applied to the data files in order to obtain the respective version [7].

Furthermore, Delta also keeps track of so-called "checkpoint" files. These are Parquet files consisting of all the actions in the JSON files up until that point in time. These checkpoint files are created periodically (the default being every 10 transactions), so that Delta can stay performant when reconstructing the current state of a table from the log files [7]. The name of the most recent checkpoint file is stored in a `_last_checkpoint` file, as depicted in Figure 2.2.

[7] `https://www.databricks.com/`

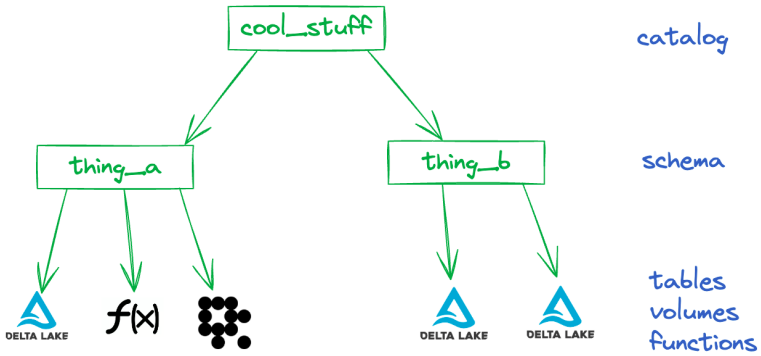

Figure 2.3: Overview of a Unity Catalog, which can be used to manage multiple Delta Lake tables [40].

Catalog

Delta Lake can be connected with a catalog, allowing the management of multiple tables and creating interoperability between other open table formats, such as Iceberg. The most popular catalog used to manage Delta tables is Unity Catalog[8]. There are two versions of this catalog: A proprietary version maintained by Databricks and an open-source version. We focus on the latter, since it is freely available. Similar to the Polaris catalog, the Unity Catalog server can be accessed through a REST API and it uses a relational database to store information about the Delta tables that it knows about.

Although a catalog server is not part of the Delta Lake specification, as it is in Iceberg's case, we include it in our research to make sure we perform a fair comparison between the systems we are analyzing (see also Section 2.5).

2.2.2 Writing to Delta tables

The process of writing data to a Delta table can be broken down into the following steps [7]:

1. Delta will first look for the `_last_checkpoint` file. Using the checkpoint file's ID, all subsequent log and checkpoint files are listed.

2. Using the listed files, the engine will read them to reconstruct the current state of the table, as well as statistics such as min and max values of columns and partitioning information.

3. Once all those files have been read, new data objects are written (possibly in parallel) to the underlying file storage.

[8] `https://www.unitycatalog.io/`

4. Then, assuming the latest log file has ID n, Delta attempts to create a new log file named $n + 1$`.json`. Since there can only be one file corresponding to a specific version of the table, creating the new log file needs to happen atomically. If this step fails due to a concurrent write operation, the transaction can be retried.

5. (Optional) Depending on the configured interval for writing checkpoints, a new checkpoint file will be created and the `_last_checkpoint` file is updated to reflect that change.

It is important to note that, like in Iceberg (Section 2.1.2), optimistic concurrency control is employed for write operations. However, Delta relies on the underlying object store's mechanism to ensure transaction atomicity [7]. For instance, Azure Blob Store, Google Cloud Storage and Amazon S3 support conditional write operations [21], [31], [33], thus ensuring that only one client can create a log file with a specific name.

2.3 DuckLake

DuckLake is an open table format introduced in 2025 as part of the DuckDB Foundation. Although it is currently an experimental release (as of writing, the current version is 0.3 [17]), we believe it will be a valuable addition to our research, as its specification proposes a new way of storing metadata and an interesting approach for handling changes to its tables.

2.3.1 Architectural design

Data layer

DuckLake's data layer consists of Parquet data files on the chosen storage system, which make up the table.

Catalog database

Similar to Iceberg, DuckLake also includes a catalog in its specification. However, DuckLake's catalog differs from Iceberg's in that it stores all of the table's metadata instead of storing the pointer to the current metadata file. In total, the catalog database contains 22 tables that together keep track of the schema, snapshots, data files, tables, statistics and partitioning information of DuckLake tables [14]. Currently, DuckLake supports DuckDB[9], SQLite[10], PostgreSQL and MySQL[11] as its catalog database [11]. Figure 2.4 shows the most relevant tables stored in the catalog [14]:

[9] `https://duckdb.org/`
[10] `https://sqlite.org/index.html`
[11] `https://www.mysql.com/`

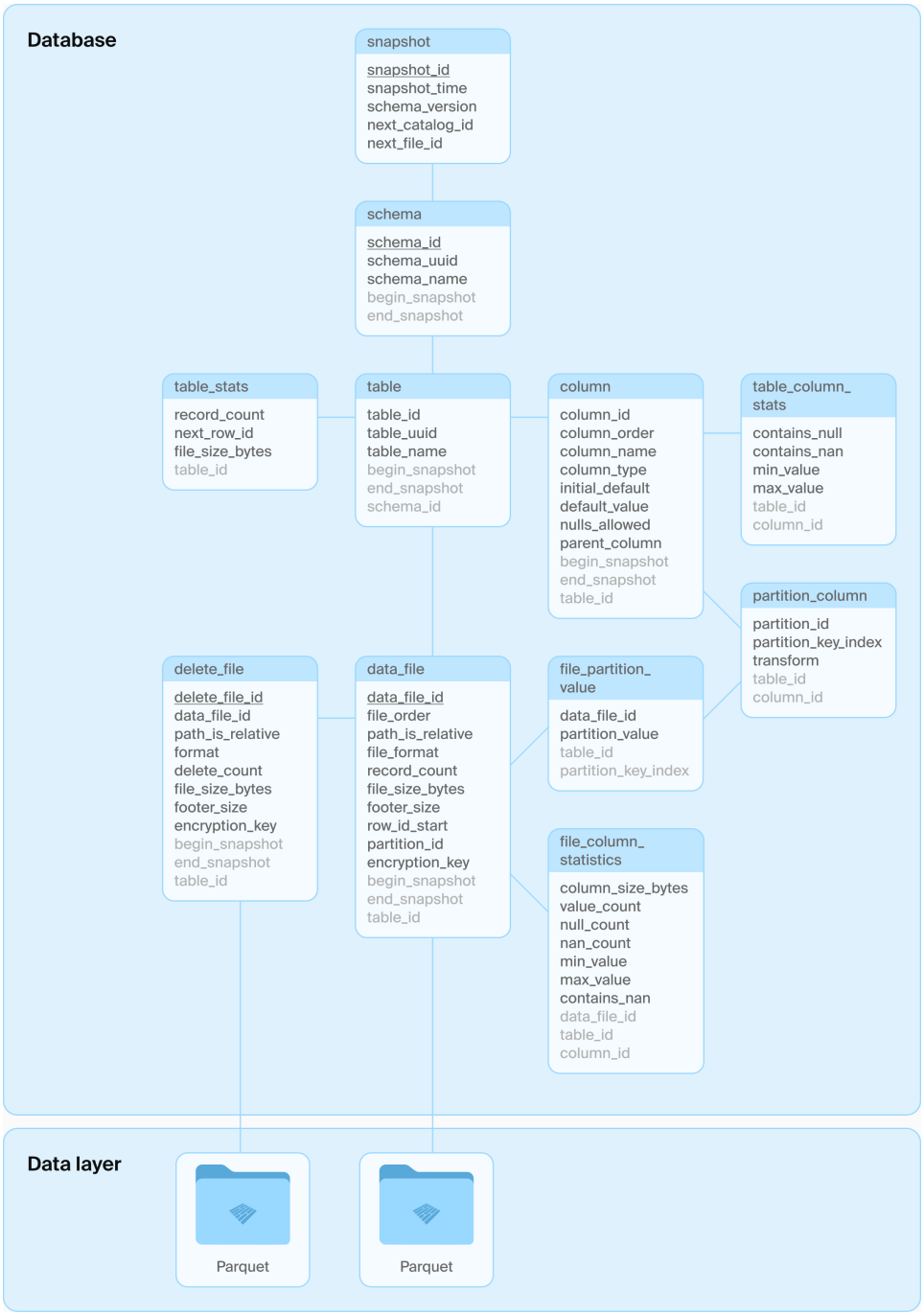

Figure 2.4: Architecture of a DuckLake table, as presented in [26].

- `table`: This is where the DuckLake tables' name, path and schema are stored. In addition, it tracks the snapshot interval in which the table is valid.
- `table_stats`: This contains metadata about the DuckLake tables, such as the number of records and the total file size of the data files that compose this table.
- `column`: This table stores the columns that are part of the DuckLake table. The main attributes are the column's name, its type and default value.
- `table_column_stats`: Similar to `table_stats`, this table contains metadata about each column in a DuckLake table, e.g. min and max values and whether the column has `NULL` values.
- `partition_column`: Contains partitioning information about columns on which partitioning has been enabled. It also stores the transformation function that is applied to the column to obtain the partition key, if such a function is required. For instance, if a table contains a timestamp column, one can apply a transformation function to that column and use the resulting value as the partitioning key.
- `data_file`: This table stores information about the data files, such as the path, format (for now, only Parquet is allowed [14]), number of records in the file and the size in bytes.
- `file_column_stats`: Similar to `table_column_stats`, this table stores column statistics on a file level, e.g. amount of records, min and max values and number of `NULL` values.
- `delete_file`: Similar to `data_file`, this table stores the path and format of delete files[12], but it contains the number of records deleted by the file.
- `snapshot`: This table is used for representing the snapshots, i.e. versions, of the DuckLake instance. Every change to a table will result in a new snapshot with a corresponding timestamp.
- `snapshot_changes`: For each snapshot, this table stores a list of changes that were made to the DuckLake instance, e.g. `CREATE`, `INSERT`, `UPDATE`, `ALTER`.

[12]Delete files are used to mark which records have been deleted from a table without actually deleting any files. This is done because Parquet files are immutable and would have to be rewritten every time a record is updated or deleted. By using delete files, deleted records can be reconciled at read time in a process called merge-on-read (MoW) [29], which is beyond the scope of this thesis.

- `schema`: This table stores the different schemas that can be used to separate DuckLake tables into namespaces. The table stores the schema name, its path in the file system, as well as the snapshot interval in which that schema is valid.

2.3.2 Writing to DuckLake

Writing to DuckLake tables happens in the following steps [26]:

1. First, the Parquet files are written to storage, unless data inlining (2.3.3) is enabled and the number of inserted rows is lower than the set threshold.

2. Then, in a single database transaction, the following tables are updated to reflect the new changes:

   - `data_file`
   - `table_stats`
   - `table_column_stats`
   - `file_column_statistics`
   - `snapshot`
   - `snapshot_changes`

   DuckLake enforces a primary key constraint on the snapshot id in the `snapshot` table [12]. If two clients try to concurrently create a new snapshot, one of them will fail due to a `PRIMARY KEY` constraint violation [12]. In that case, DuckLake will try to solve the conflict without having to rewrite the data files. For that, it looks at the `snapshot_changes` table to see what changes were made by the transaction that succeeded. If there are no logical conflicts, e.g. both clients just added new data, the failed client can retry the transaction without having to rewrite the data files [12].

2.3.3 Data inlining

One feature that sets DuckLake apart from Delta and Iceberg is the so-called "data inlining". This feature allows DuckLake to be configured such that writes that insert fewer rows than a set threshold are stored in the catalog database, rather than written out as Parquet files [13]. This can increase the performance of DuckLake for frequent, small, updates. The "inlined" data is immediately visible for read operations and it can be flushed out to Parquet files once enough data has been accumulated in the catalog [13].

2.4 Data streams

A data stream can be defined as data that is produced by one or more sources continuously and thus, is considered unbounded [15], [39]. Examples of data streams are measurements produced by IoT sensors, web server logs and network traffic data. Another key characteristic of stream data is the presence of at least one timestamp, which is assigned when the data point, i.e. event, is produced or processed by a system [15].

2.5 Benchmarking

When it comes to evaluating systems' performance, it is crucial that the comparison is done fairly to avoid misleading results. In that regard, Raasveldt et al. [30] highlight some of common pitfalls when benchmarking database systems. These pitfalls were obtained from a literature review on best practices and recommendations for both benchmarking in general and benchmarking of database management systems (DBMSs). The authors then identified seven[13] different pitfalls that can occur during DBMS benchmarking, namely:

1. **Reporting non-reproducible results**: In scientific research in general, it is required that results are reproducible, so that they can be independently verified. Benchmarking is no exception to this rule.

2. **Sub-optimally optimizing systems**: In cases where the authors of a paper are comparing their own system with the state of the art, they might feel tempted to not properly optimize the systems they are comparing with, so that they get favorable results [30]. In some other cases, failure to properly optimize a system might happen due to lack of familiarity with that system.

3. **Overly-optimizing a system**: Similarly to the previous pitfall, one might optimize their system to perform really well on a specific benchmark. This can be done by optimizing a system for the specific dataset or workloads present in a benchmark, which are all known prior to execution.

4. **Benchmarking code that contains bugs**: Sometimes, performance measurements can be influenced by bugs in the SUTs or in the code performing the benchmark. Bugs can lead to an unfair advantage by skipping a step of the benchmark workload or by only working with a specific dataset [30].

[13] In the original paper, the authors report "cold" vs "hot" runs and "cold" vs "warm" runs separately [30] , so they reported eight pitfalls in total. We believe however that they pertain to the same aspect of benchmarking, so we include them all in the same pitfall.

5. **Comparing systems that are fundamentally different**: Benchmark results can only be considered fair if they were obtained from systems that provide the same functionality. Otherwise, one of the systems might be given an unfair advantage by avoiding a set of constraints present in other systems. Raasveldt et al. [30] exemplify this by comparing a complete DBMS with an algorithm specialized in one task. The algorithm has an advantage because it takes fewer aspects into account than the DBMS [30].

6. **Ignoring preprocessing time**: When running benchmarks, the SUTs often have to be initialized with the dataset used in benchmark's workload. This initialization can sometimes include preprocessing steps that a system executes to speed up performance of subsequent tasks. The example given in [30] mentions the creation of database indexes, which if ignored, might favor a system that has efficient, but slow-to-create, indexes.

7. **Comparing "cold", "warm" and "hot" runs**: The concept of a "cold" run refers to executing a benchmark for the first time on a system for which no data has been cached on the host operating system yet. By contrast, subsequent runs of the benchmark are considered "hot", due to the creation of caches and other optimizations that the underlying operating systems might execute. Raasveldt et al. [30] also alert for "warm" runs, which can occur when the SUT is restarted to perform a "cold" run, but caches are still present in the OS memory [30].

In addition, the authors in [30] suggest practices on how to prevent these pitfalls from happening. In some cases, such as that of non-reproducible experiments, it can be easily avoided by including a detailed specification of the configuration parameters used, hardware specifications and source code. However, other pitfalls are not so trivial to avoid. For instance, making sure the SUTs are not running sub-optimally can be hard depending on the amount of configuration available for each system. Nonetheless, we will follow the checklist provided in [30, Appendix A] and mention it whenever relevant throughout this thesis.

Chapter 3

Related Work

3.1 Benchmarking lakehouses

Lakehouses are a novel technology when compared to the more established DBMSs like PostgreSQL or MySQL. Therefore, not many studies have been published evaluating the performance of open table formats. For that reason, we focus on two notable works in this section: LHBench [29] and LST-Bench [9].

The authors in [29] leverage the TPC-DS benchmark [38] to evaluate three lakehouse systems, namely Apache Iceberg, Delta Lake and Apache Hudi[1]. Although the authors focused on query performance of lakehouses when reading data, they also presented some results for insert operations. They looked at the creation time of TPC-DS tables with 3 TB of bulk data. Their results show that Iceberg and Delta have similar loading times around 2,700 and 2,300 seconds respectively, whereas Hudi takes around 20,000 seconds to load all the data. They attribute this difference to Hudi performing more preprocessing at insertion than Delta and Iceberg. The authors performed a second experiment where they loaded 100 GB of TPC-DS data into each lakehouse and the results were proportional to the first experiment: Delta took around 420 seconds, Iceberg took around 320 seconds and Hudi took upwards of 2,400 seconds to load all the data. It is important to stress that these experiments were run on Spark[2] clusters of 16 workers, each with 8 virtual CPUs and 61 GB of memory [29].

Camacho-Rodríguez et al. [9] presented LST-Bench, which also uses the TPC-DS benchmark, but extends it to include workloads that measure specific characteristics of lakehouses. For instance, the authors looked at the impact that merging many data files into fewer, larger files had on read performance. They also looked at how *not* performing those merge operations

[1] Hudi is another open table format that we excluded from this thesis due to the lack of a dedicated catalog for it.

[2] `https://spark.apache.org/`

can degrade metrics such as CPU usage, memory consumption and disk I/O. Their findings show that merging data files significantly helps maintain performance of Delta Lake and Iceberg, whereas Hudi, again, did not benefit so much from those merge operations. This result agrees with [29], as discussed previously. Furthermore, the results of LST-Bench showed that not merging data files can degrade the performance up to 6.8× [9]. Finally, they also utilized clusters of 17 nodes, where each node consisted of 8 virtual CPUs and 64 GB of memory [9].

Both papers, though quite comprehensive in their analyses, do not investigate the performance of writing streams of data into lakehouses. To the best of our knowledge, no other work has been done on that regard either. Therefore, we will address that gap in this thesis.

3.2 Common streaming benchmark patterns

Although our work does not focus on SPEs, we go over relevant characteristics of SPE benchmark systems, so that we can draw ideas for designing our own benchmark.

Yue et al. [43] performed an extensive survey of benchmarks of SPEs, focusing on four characteristics: (i) Dataset type, (ii) data ingestion method, (iii) SUT, (iv) metrics collected. We focus here on dataset type, the data ingestion method and metrics collected. The SUTs mentioned in [43] are not relevant, since our thesis does not look at the performance of SPEs.

3.2.1 Dataset type

As Yue et al. [43, Table 2] show in their paper, datasets are classified into either synthetic or real-world data. Their results indicate that there is a relatively even usage of both types of datasets. In cases where a synthetic dataset is used, e.g. the Linear Road benchmark [6], it is due to the benchmark workload requiring a specific schema for the data. Another example, though not mentioned in [43], is the PGVal benchmark [34], which required a deterministic dataset of web server logs, so the authors had to generate their own data.

In terms of real-word data, Yue et al. [43] found a variety of datasets in the compiled benchmarks. These datasets include IoT sensor data, Twitter data, network traffic data, e-commerce data, online game data, etc.

As discussed in Section 2.4, there are few requirements that characterize data streams. Therefore, many datasets are suitable for simulating streams of data.

3.2.2 Data ingestion

The vast majority of benchmarks gathered in [43] make use of Apache Kafka[3] to ingest data. Kafka is a "distributed event streaming platform" [5] that acts as an event broker between data producers, e.g. an IoT sensor, and consumers, such as Apache Spark, Apache Flink[4] or client APIs for various programming languages. An event is always written to a Kafka topic, so that different types of events can be easily organized and consumers can consume events from the topics they are interested in.

The few papers compiled in [43] that do not use Kafka for data ingestion mostly use a self-designed ingestion method. For instance, Linear Road [6] uses the MIT Traffic Simulator [42], which generates data into flat files that are read by a data driver to simulate incoming traffic data.

In other cases, such as in [25], the authors argue that using an event broker such as Kafka can lead to bottlenecks in the benchmark pipeline. In short, they suggest that the process of storing and partitioning incoming data by the broker can slow down the rate at which SPEs can consume that data [25]. They instead use an in-memory generator, which they tuned to make sure it can produce data faster than the fastest consumer in their benchmark [25]. However, they do not provide any source code for their generator, nor do they supply any data comparing Kafka with their own generator, so it is unclear whether Kafka would have been a bottleneck in their system.

3.2.3 Metrics

In terms of performance metrics compiled by Yue et al. [43], most benchmarks focus on throughput (Section 3.2.4), processing latency, CPU/memory/network usage and disk I/O. Some benchmarks focus on other aspects of SPEs, such as processing guarantees [34] and scalability capacities [19].

In this thesis, we focus primarily on throughput of data streams, which is the main objective of our research question. In addition, we collect CPU usage and disk I/O metrics for a more fine-grained overview of the systems' resource utilization. We do not look at latency, since no processing will be done on the data streams before writing them to the lakehouses. Further, we will not look at memory usage because we will not be writing large amounts of data at once. Rather, we will look at small, constant writes, so we expect memory consumption to be relatively low. Finally, we will not collect network usage metrics, since our experiments will be run locally. We believe this might help to more accurately measure throughput because having a network connection to the storage layer would add noise to our results when writing events in the lakehouses.

[3] `https://kafka.apache.org/`
[4] `https://flink.apache.org/`

3.2.4 Throughput

When benchmarking streaming processing engines (SPEs), a common metric used is the system's throughput, which is defined as the amount of events that the system under test (SUT) can process in a given unit of time. In terms of execution time, Henning et al. [20] presented a thorough approach for measuring throughput, where they run their experiments for 15 minutes and repeat it three times. In addition, Henning et al. [19] run their experiments for five minutes, but the first minute is considered "warm up", which relates to the idea of "cold" vs "warm" runs discussed in [30]. Both of these approaches will be adapted for our experiments.

Chapter 4

Methodology

In this chapter we discuss the implementation of our benchmarking tool. We go over the data ingestion process, data formats used and the metrics that we collect. We also give an overview of the experimental setup, making sure to include as many details to facilitate reproducibility of our results.

4.1 Benchmark design

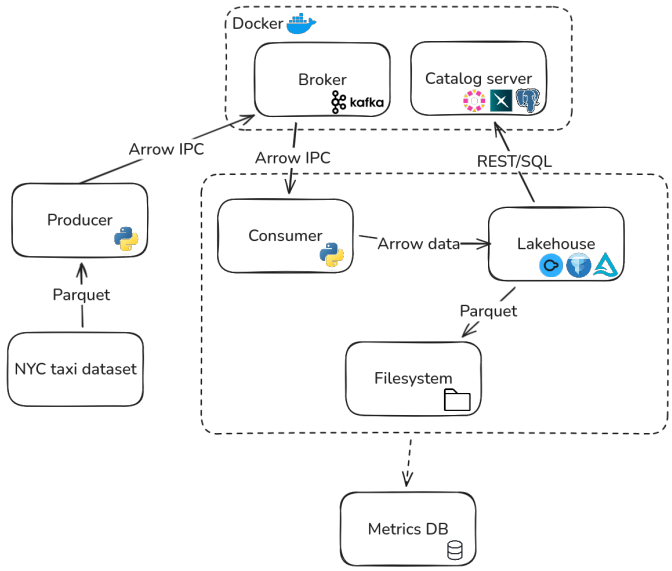

Figure 4.1: Overview of the components in our benchmarking tool.

4.1.1 Dataset

Our benchmark uses the TLC Trip Record Data [36], which is a publicly available dataset of taxi trips in New York City. We use specifically the

trip data of yellow taxi cabs during November 2025, totaling 4,181,444 trip records. This dataset provides a good representation of real-world stream data, since taxi trips are constantly occurring. Further, the dataset can be easily extended to include data from previous years, in case there is the need to scale up the benchmark.

As a preprocessing step, we added a single column to the dataset with an integer id for each trip. This helps us track statistics for each trip processed by our system and it also increases reproducibility of our results.

4.1.2 Data ingestion

Following the approach of previous works, we ingest the dataset into a Kafka broker instance, using the Kafka Python client API[1]. For reproducibility purposes, the broker runs in a Docker container. A single topic is created for the events being produced and we delete that topic after every experiment, to ensure we do not mix events from different runs. The Kafka producer runs in a separate Python process, so that our benchmark can consume the Kafka events independently.

4.1.3 Data format

We use the Arrow format for data representation in our benchmark. Apache Arrow[2] is a standardized, in-memory columnar format, which facilitates data exchange between systems implementing it. The columnar layout of Arrow data also makes it very efficient for writing it in Parquet format [1]. Furthermore, the Arrow specification includes an inter-process communication (IPC) mechanism [2], which can be used to stream arrow data while avoiding the use of serialization or extra copies to/from the in-memory Arrow layout. Therefore, we use the IPC mechanism for producing data to the Kafka broker, so that the consumer can read it as Arrow data directly.

4.1.4 Lakehouses

Our benchmark can interact with the different lakehouse systems in two ways. The default way uses a dedicated library[3] for each system, which is optimized for that specific lakehouse format and is compatible with Arrow data. Alternatively, the benchmark can be run with DuckDB as the client for all three lakehouse formats, since it has first-class support for all three formats [10]. In both cases, the client used to interact with the lakehouse is considered part of SUT and is reported explicitly for clarity of results.

[1] `https://docs.confluent.io/platform/current/clients/confluent-kafka-python/html/index.html`

[2] `https://arrow.apache.org/`

[3] PyIceberg, delta-rs and DuckDB

Additionally, all catalog servers are run on Docker containers, so that the benchmark can be run on any machine, thus contributing to reproducibility of experiments. The only exception is when using DuckDB as a catalog for DuckLake. In that case, the catalog is stored in a local file. This combination is only used when running experiments that involve the "data inlining" feature (Section 2.3.3).

4.1.5 Metrics

All collected metrics are stored in a local database for further analysis. The throughput is calculated as the number of events written to the given lakehouse system, divided by the total number of seconds that the experiment was executed. In addition, the following metrics are collected:

- Time taken to read each event from the Kafka broker: We expect read times to be low and consistent, since our benchmark reads events one at a time. Nonetheless, we collect this data to ensure that reading the data does not cause any bottlenecks in our experiments.
- Time taken to write each event to the lakehouse table: On top of looking at the throughput, we analyze the individual writing time of each event to see if systems can keep consistent writing times.
- CPU usage: We run a separate Python thread that collects the total CPU usage percentage every second. For that, we use the `psutil` [16] Python library.
- Disk I/O: Similar to CPU usage, we collect the amount of bytes written to the machine's main disk at a one-second interval. `psutil` returns the total amount of bytes written since system startup, which strictly increases over time. Instead, we focus on the difference in bytes written between every second, so we can relate the average number of bytes written per second with the throughput of each system.

4.2 Experimental setup

Combining the approaches in [19], [20], we perform three runs of five minutes for each lakehouse, where we consider the first run to be cold and the others to be hot. To ensure cold runs, we restart the machine running the experiments after the three runs for each lakehouse.

Regarding the lakehouse systems evaluated, we only evaluate Iceberg and DuckLake. During the setup of our experiments, we encountered a bug in the `delta-rs` library, which prevents access to tables through Unity Catalog. We opened a GitHub issue[4] for this, but as of writing this thesis, the problem

[4] `https://github.com/delta-io/delta-rs/issues/3966`

has not been solved yet. Although it is still possible to access and write to Delta tables without Unity Catalog, we believe this would give an unfair advantage to Delta, since the other two systems are connected to catalogs.

4.2.1 Configuration

For DuckLake and Iceberg we use the default configuration of each system, except for the Parquet compression codec and the number of threads used by each system. We set the former to use the snappy [28] algorithm, which provides fast compression speeds. We explicitly set the number of threads to 1, since our benchmark inserts a single event in the SUT each time. Even in the case of Iceberg, where multiple metadata files are written at every insert, these files must be written sequentially, because each metadata file holds a reference to another data- or metadata file below it (Figure 2.1). We confirmed there is no benefit to using multiple threads by running some initial tests, where we noticed no increase in the number of events inserted to the lakehouse table when going from 1 to 16 threads. In fact, in the case of DuckLake, there was a significant decrease in the number of events inserted, as shown in Figure 4.2.

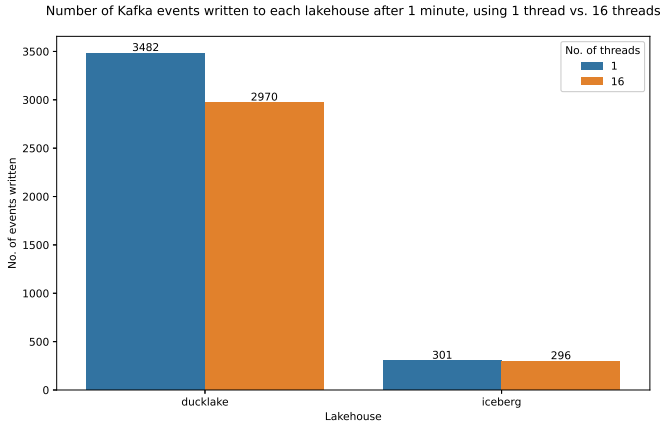

Figure 4.2: Overview of the components in our benchmarking tool.

In addition, we perform runs using the "native" client for each lakehouse, DuckDB for DuckLake and PyIceberg for Iceberg, as well as using DuckDB for both. This will give us insight into how much of the performance is impacted by the client used for interacting with the lakehouse tables. PyIceberg does not have the option of interacting with other open table formats, so we cannot use it to write to DuckLake.

Finally, when running experiments with the inlining feature of DuckLake enabled, we set the write threshold to be 2 elements, since we are writing events one by one to the table.

4.2.2 Hardware

All experiments were executed on a local machine for which the hardware specification can be found in Table 4.1. We believe that running experiments locally significantly reduces complexity and noise that would otherwise be present in a cloud environment.

OS	Linux Mint 22.3 - Cinnamon 64-bit
CPU	AMD Ryzen 7 5700U 1.8 GHz base clock 8 cores and 16 threads
Memory	2× 8 GB DDR4 @ 1600 MHz
SSD	Intel 660p 1 TB NVMe M.2 Up to 1,800 MB/s for sequential writes Up to 220,000 IOPS for random writes

Table 4.1: Overview of the hardware specs used for running our benchmark

Chapter 5

Results

Chapter 6

Discussion

6.1 Limitations

Chapter 7

Conclusion and Future Work

Bibliography

[1] *Apache Arrow - Frequently Asked Questions*, English. Accessed: Feb. 2, 2026. [Online]. Available: https://arrow.apache.org/faq/

[2] *Apache Arrow - Streaming, Serialization, and IPC*, English. Accessed: Feb. 3, 2026. [Online]. Available: https://arrow.apache.org/docs/python/ipc.html

[3] Apache Iceberg, *Reliability*, English. Accessed: Oct. 22, 2025. [Online]. Available: https://iceberg.apache.org/docs/latest/reliability/

[4] Apache Iceberg, *REST Catalog Spec*, en, Oct. 2025. Accessed: Oct. 15, 2025. [Online]. Available: https://iceberg.apache.org/rest-catalog-spec/

[5] *Apache Kafka*, Nov. 2025. [Online]. Available: https://github.com/apache/kafka

[6] A. Arasu et al., "Linear road: A stream data management benchmark," in *Proceedings of the thirtieth international conference on very large data bases - volume 30*, ser. Vldb '04, Number of pages: 12, Toronto, Canada: VLDB Endowment, 2004, pp. 480–491, ISBN: 0-12-088469-0.

[7] M. Armbrust et al., "Delta lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, Aug. 2020, ISSN: 2150-8097. DOI: 10.14778/3415478.3415560 Accessed: Jun. 27, 2025. [Online]. Available: https://dl.acm.org/doi/10.14778/3415478.3415560

[8] Avril Aysha, *Delta Lake vs Data Lake - What's the Difference?* English. Accessed: Jan. 22, 2026. [Online]. Available: https://delta.io/blog/delta-lake-vs-data-lake/

[9] J. Camacho-Rodríguez et al., "LST-Bench: Benchmarking Log-Structured Tables in the Cloud," en, *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–26, Mar. 2024, ISSN: 2836-6573. DOI: 10.1145/3639314 Accessed: Sep. 8, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3639314

[10] *DuckDB Documentation - Lakehouse Formats*, English, Oct. 2025. Accessed: Feb. 2, 2026. [Online]. Available: https://blobs.duckdb.org/docs/duckdb-docs.pdf

[11] *Ducklake Documentation - Choosing a Catalog Database*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/usage/choosing_a_catalog_database

[12] *DuckLake Documentation - Conflict Resolution*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/advanced_features/conflict_resolution

[13] *DuckLake Documentation - Data Inlining*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/advanced_features/data_inlining

[14] *Ducklake Documentation - Tables*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/specification/tables/overview

[15] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, "A survey on the evolution of stream processing systems," en, *The VLDB Journal*, vol. 33, no. 2, pp. 507–541, Mar. 2024, ISSN: 1066-8888, 0949-877X. DOI: 10.1007/s00778-023-00819-8 Accessed: Sep. 15, 2025. [Online]. Available: https://link.springer.com/10.1007/s00778-023-00819-8

[16] Giampaolo Rodola, *Psutil*, Jan. 2026. [Online]. Available: https://github.com/giampaolo/psutil

[17] Guillermo Sanchez and Gabor Szarnyas, *DuckLake 0.3 with Iceberg Interoperability and Geometry Support*, English, Sep. 2025. Accessed: Nov. 21, 2025. [Online]. Available: https://ducklake.select/2025/09/17/ducklake-03/

[18] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," en, *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, Dec. 1983, ISSN: 0360-0300, 1557-7341. DOI: 10.1145/289.291 Accessed: Jan. 22, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/289.291

[19] S. Henning and W. Hasselbring, "Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures," en, *Big Data Research*, vol. 25, p. 100 209, Jul. 2021, ISSN: 22145796. DOI: 10.1016/j.bdr.2021.100209 Accessed: Sep. 15, 2025. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2214579621000265

[20] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, "ShuffleBench: A Benchmark for Large-Scale Data Shuffling Operations with Distributed Stream Processing Frameworks," en, in *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, London United Kingdom: ACM, May 2024, pp. 2–13, ISBN: 979-8-4007-0444-4. DOI: 10.1145/3629526.3645036 Accessed: Sep. 15, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3629526.3645036

[21] *How to prevent object overwrites with conditional writes*, English. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html

[22] *Iceberg*, Dec. 2025. [Online]. Available: https://github.com/apache/iceberg

[23] *Iceberg Spec - Optimistic Concurrency*, English. Accessed: Jan. 26, 2026. [Online]. Available: https://iceberg.apache.org/spec/?h=concurren#optimistic-concurrency

[24] A. M. Jason Hughes, *Apache Iceberg: The Definitive Guide*, eng. O'Reilly Media, Inc, 2024, OCLC: 1425185829, ISBN: 978-1-0981-4861-4.

[25] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking Distributed Stream Data Processing Systems," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, Paris: IEEE, Apr. 2018, pp. 1507–1518, ISBN: 978-1-5386-5520-7. DOI: 10.1109/ICDE.2018.00169 Accessed: Sep. 15, 2025.

[26] Mark Raasveldt and Hannes Mühleisen. "The DuckLake manifesto: SQL as a lakehouse format," DuckLake, Accessed: Jun. 27, 2025. [Online]. Available: https://ducklake.select/manifesto/

[27] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," presented at the CIDR, 2021. Accessed: Jun. 26, 2025. [Online]. Available: https://vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html

[28] opensource@google.com, *Snappy*, Mar. 2025. [Online]. Available: https://github.com/google/snappy

[29] Paras Jain, Peter Kraft, Conor Power, Tathagata Das, Ion Stoica, and Matei Zaharia, "Analyzing and comparing lakehouse storage systems," presented at the CIDR, Amsterdam, The Netherlands, 2023. [Online]. Available: https://vldb.org/cidrdb/2023/analyzing-and-comparing-lakehouse-storage-systems.html

[30] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing," en, in *Proceedings of the Workshop on Testing Database Systems*, Houston TX USA: ACM, Jun. 2018, pp. 1–6, ISBN: 978-1-4503-5826-2. DOI: 10.1145/3209950.3209955 Accessed: Sep. 8, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3209950.3209955

[31] *Request preconditions*, English, Jan. 2026. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/request-preconditions

[32] J. Schneider, C. Gröger, A. Lutsch, H. Schwarz, and B. Mitschang, "The lakehouse: State of the art on concepts and technologies," *SN Computer Science*, vol. 5, no. 5, p. 449, Apr. 18, 2024, ISSN: 2661-8907. DOI: 10.1007/s42979-024-02737-0 Accessed: Jun. 23, 2025. [Online]. Available: https://link.springer.com/10.1007/s42979-024-02737-0

[33] *Specifying conditional headers for Blob service operations*, English, Nov. 2025. Accessed: Jan. 26, 2026. [Online]. Available: https://learn.microsoft.com/en-us/rest/api/storageservices/specifying-conditional-headers-for-blob-service-operations

[34] J. Tahir, R. Mayer, C. Doblander, and H.-A. Jacobsen, "How Reliable are Streams? End-to-End Processing-Guarantee Validation and Performance Benchmarking of Stream Processing Systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 585–598, Nov. 2024, ISSN: 2150-8097. DOI: 10.14778/3712221.3712227 Accessed: Sep. 18, 2025.

[35] Tathagata Das and Denny Lee, *Delta 2.0 - The Foundation of your Data Lakehouse is Open*, English, Aug. 2022. Accessed: Oct. 22, 2025. [Online]. Available: https://delta.io/blog/2022-08-02-delta-2-0-the-foundation-of-your-data-lake-is-open/

[36] Taxi & Limousine Comission, *TLC Trip Record Data*, English. Accessed: Feb. 3, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[37] The Delta Lake Project Authors, *Delta Lake*, Jan. 2026. Accessed: Jan. 23, 2026. [Online]. Available: https://github.com/delta-io/delta

[38] *TPC BENCHMARK DS*, English, Nov. 2024. Accessed: Sep. 11, 2025. [Online]. Available: https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-DS_v4.0.0.pdf

[39] Tyler Akidau, *Streaming 101: The world beyond batch*, English, Aug. 2015. Accessed: Nov. 12, 2025. [Online]. Available: https://www.oreilly.com/radar/the-world-beyond-batch-streaming-101/

[40] *Unity Catalog Structure*, English, Jul. 2024. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.unitycatalog.io/quickstart/

[41] *What is a data lake?* English. Accessed: Jan. 22, 2026. [Online]. Available: https://www.cloudflare.com/learning/cloud/what-is-a-data-lake/

[42] Q. Yang and H. N. Koutsopoulos, "A Microscopic Traffic Simulator for evaluation of dynamic traffic management systems," en, *Transportation Research Part C: Emerging Technologies*, vol. 4, no. 3, pp. 113–129, Jun. 1996, ISSN: 0968090X. DOI: 10.1016/S0968-090X(96)00006-X Accessed: Jan. 30, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0968090X9600006X

[43] W. Yue, M. Boissier, and T. Rabl, "A Survey of Stream Processing System Benchmarks," en, in *Performance Evaluation and Benchmarking*, R. Nambiar and M. Poess, Eds., vol. 15337, Series Title: Lecture Notes in Computer Science, Cham: Springer Nature Switzerland, Jul. 2025, pp. 24–43, ISBN: 978-3-031-93857-3 978-3-031-93858-0. DOI: 10.1007/978-3-031-93858-0_2 Accessed: Sep. 12, 2025. [Online]. Available: https://link.springer.com/10.1007/978-3-031-93858-0_2

Appendix A